

Intercity Carpooling App

Ride Start, Pickup, Drop-off, Live Tracking, Dispute & Completion Design Document

Scope: Post-booking operational flow. Ride publish and booking are already completed.

1. 1. Purpose and Scope
2. 2. Business Scenario
3. 3. High-Level Architecture
4. 4. Core Domain Model
5. 5. State Machines
6. 6. End-to-End Ride Flow
7. 7. OTP Pickup Verification
8. 8. Live Tracking and Family Sharing
9. 9. Offline and Server Down Handling
10. 10. Payments and Payout Eligibility
11. 11. Dispute Review System
12. 12. Edge Cases
13. 13. API Design
14. 14. Database Model
15. 15. Frontend Screens and Wireframes
16. 16. Admin Dashboard
17. 17. MVP Scope and Future Enhancements
18. 18. Development Checklist

1. Purpose and Scope

This document defines the complete development design for the ride operation lifecycle of an intercity carpooling application. It starts after a ride has already been published and booked. The main goal is to allow the driver to start the ride, pick up riders using OTP verification, perform drop-offs, support live tracking, handle offline/server failure, manage disputes, and make payments payout eligible safely.

- Ride publishing and booking are out of scope and assumed complete.
- Each rider booking has its own pickup, onboard, drop-off, dispute, and payment lifecycle.
- The main ride can be completed only after all bookings are in a terminal state.
- Payments are handled per booking so one disputed rider does not block the whole driver payout.

2. Business Scenario

Example route used throughout the document:

Stop 1 -> Stop 2 -> Stop 3 -> Stop 4

Rider	Pickup	Drop-off	Segment
Rider 1	Stop 1	Stop 3	Stop 1 -> Stop 2 -> Stop 3
Rider 2	Stop 2	Stop 4	Stop 2 -> Stop 3 -> Stop 4
Rider 3	Stop 2	Stop 4	Stop 2 -> Stop 3 -> Stop 4

3. High-Level Architecture

The architecture separates ride state, booking state, location tracking, payments, OTP, notifications, and dispute handling. Location data is kept separate from core ride data to reduce write pressure on the main database.

flowchart TD

```
DriverApp[Driver App] --> APIGW[API Gateway]
RiderApp[Rider App] --> APIGW
FamilyWeb[Family Live Tracking Web] --> APIGW
AdminWeb[Admin Dashboard] --> APIGW
```

```
APIGW --> RideService[Ride Service]
APIGW --> BookingService[Booking Service]
APIGW --> LocationService[Location Tracking Service]
APIGW --> OTPService[OTP Service]
APIGW --> TrackingLinkService[Tracking Link Service]
APIGW --> DisputeService[Dispute Service]
```

```
RideService --> RideDB[(Ride DB)]
BookingService --> BookingDB[(Booking DB)]
LocationService --> LatestLocation[(Latest Location Store)]
LocationService --> LocationHistory[(Location History Store)]
OTPService --> OTPStore[(OTP Store)]
```

```
BookingService --> GeofenceService[Geofence Service]
BookingService --> PaymentService[Payment Service]
BookingService --> NotificationService[Notification Service]
```

```
PaymentService --> PaymentProvider[Payment Provider]
NotificationService --> PushProvider[Push/SMS/Email]
DisputeService --> EvidenceCollector[Evidence Collector]
EvidenceCollector --> LocationHistory
EvidenceCollector --> BookingDB
EvidenceCollector --> RideDB
```

Service	Responsibility
Ride Service	Owns ride lifecycle: start, in-progress, completion pending, completed, cancelled.
Booking Service	Owns each rider booking lifecycle: pickup, OTP verification, onboard, drop-off, no-show, missed pickup.
OTP Service	Generates, stores, validates, expires and consumes OTP per

	booking.
Location Tracking Service	Receives driver GPS updates every 5-10 seconds and powers live tracking.
Geofence Service	Validates if driver/rider actions happen near expected pickup/drop-off coordinates.
Payment Service	Keeps funds in escrow, applies payout/refund decisions, releases payouts after dispute window.
Dispute Service	Collects evidence, auto-resolves clear cases, queues unclear cases for admin review.
Offline Sync Service	Processes queued mobile actions with idempotency keys when connectivity returns.
Notification Service	Sends push/SMS/email updates for ride started, driver arrived, pickup, drop-off, disputes.
Tracking Link Service	Creates secure live tracking links for rider family members.

4. Core Domain Model

Entity	Purpose	Important Fields
Ride	Overall driver journey.	id, driverId, status, routeId, actualStartTime, actualEndTime, currentStopSequence
Stopover	Ordered route stop.	id, rideId, sequence, lat, lng, plannedArrivalTime, geofenceRadiusMeters
Booking	A rider seat from pickup stop to drop-off stop.	id, rideId, riderId, pickupStopId, dropoffStopId, status, paymentStatus
RideEvent	Audit log of every critical action.	eventType, actorId, actionId, lat, lng, clientTimestamp, serverTimestamp, validationStatus
LocationUpdate	GPS tracking data.	rideId, driverId, lat, lng, speed, heading, accuracy, timestamp
OTP	Pickup verification token.	bookingId, otpHash, expiresAt, usedAt, attemptCount, status
Dispute	Conflict review record.	bookingId, raisedBy, reason, evidence, status, resolution
TrackingLink	Family tracking access.	bookingId, tokenHash, expiresAt, accessScope

5. State Machines

5.1 Ride States

SCHEDULED -> READY_TO_START -> IN_PROGRESS -> COMPLETION_PENDING -> COMPLETED
 |
 -> CANCELLED
 -> DISPUTED

Ride State	Meaning
SCHEDULED	Ride exists and has active bookings, but start time is not yet near.
READY_TO_START	Ride can be started by the driver.
IN_PROGRESS	Driver has started ride and live tracking is active.
COMPLETION_PENDING	Driver finished ride; backend is finalizing booking/payment states.
COMPLETED	All bookings are terminal and ride is closed.
CANCELLED	Ride was cancelled.
DISPUTED	Ride has serious active dispute; normal completion may be blocked.

5.2 Booking States

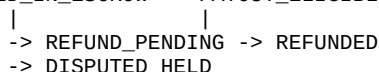
```
BOOKED -> WAITING_FOR_PICKUP -> DRIVER_ARRIVED -> OTP_PENDING -> ONBOARD -> DROP_PENDING -> DRIVER_DROPPED
-> COMPLETED
```



Booking State	Meaning
BOOKED	Booking is confirmed and paid/authorized.
WAITING_FOR_PICKUP	Ride has started and rider is expected at pickup.
DRIVER_ARRIVED	Driver reached pickup point and wait timer can start.
OTP_PENDING	Driver is verifying rider OTP.
ONBOARD	Rider successfully boarded.
DROP_PENDING	Rider is onboard and pending drop-off.
DRIVER_DROPPED	Driver marked rider dropped, waiting for rider confirmation if needed.
COMPLETED	Booking successfully completed.
NO_SHOW	Rider did not show after driver waited at pickup.
DRIVER_MISSED_PICKUP	Driver skipped or failed to pick up rider.
DISPUTED	Payment is held and dispute review is active.

5.3 Payment States

AUTHORIZED OR PAID -> HELD IN ESCROW -> PAYOUT ELIGIBLE -> PAYOUT RELEASED



6. End-to-End Ride Flow

flowchart TD

```
A[Ride already published and booked] --> B[Ride ready to start]
B --> C[Driver taps Start Ride]
C --> D[Server available?}
D -- Yes --> E[Ride Service marks ride IN_PROGRESS]
D -- No --> F[Driver app stores Start Ride action offline]
F --> G[Sync later using actionId]
G --> E
E --> H[Location tracking starts]
H --> I[Driver GPS every 5-10 seconds]
I --> J[Rider/family live tracking]
E --> L[Driver follows stop timeline]
L --> M[Driver arrives at pickup]
M --> N[Driver taps Arrived]
N --> O[Pickup geofence validation]
O --> P[Rider shows OTP]
P --> Q[Driver enters OTP]
Q --> R{OTP valid?}
R -- Yes --> S[Booking ONBOARD]
R -- No --> T[Retry / support]
S --> U[Driver continues route]
U --> V[Driver reaches drop-off]
V --> W[Driver confirms drop-off]
W --> X[Drop-off geofence validation]
X --> Y[Rider confirms drop-off]
Y --> Z{Valid?}
Z -- Yes --> AA[Booking COMPLETED]
Z -- No --> AB[Booking DISPUTED]
AA --> AC[Dispute window]
AC --> AD{Dispute raised?}
AD -- No --> AE[PAYOUT_ELIGIBLE]
AD -- Yes --> AF[DISPUTED_HELD]
AF --> AG[Dispute review]
AE --> AH{All bookings terminal?}
AB --> AH
AG --> AH
AH -- No --> L
AH -- Yes --> AI[Driver taps Finish Ride]
AI --> AJ[Ride COMPLETED]
```

7. OTP Pickup Verification

OTP is used as the main proof that the rider actually boarded the correct vehicle. OTP is per booking, not per ride, because multiple riders can board at the same stop.

19. Rider opens the app near pickup time and sees their OTP.
20. Driver arrives and taps Arrived at Pickup.
21. Driver asks rider for OTP.
22. Driver enters OTP into the driver app.
23. Backend verifies OTP, booking state, driver authorization, and pickup geofence.
24. If valid, booking becomes ONBOARD and payment remains held in escrow.

OTP Rule	Description
Per booking	Each booking gets a separate OTP. Multiple riders at same pickup must be verified separately.
Short-lived	OTP should be valid only near pickup time or until the pickup is completed.
One-time use	OTP is consumed after successful pickup verification.
Limited retries	Too many wrong attempts should lock OTP and require support/manual verification.
Geofence aware	OTP success outside pickup geofence should create a warning or review flag.

8. Live Tracking and Family Sharing

```
sequenceDiagram
    actor Driver
    actor Family
    participant DriverApp
    participant LocationService
    participant TrackingLinkService
    participant FamilyWeb

    DriverApp->>LocationService: SendLocation(rideId, lat, lng, speed, heading, timestamp)
    LocationService->>LocationService: Store latest location
    Family->>FamilyWeb: Open secure tracking link
    FamilyWeb->>TrackingLinkService: Validate token
    TrackingLinkService->>LocationService: Get latest ride location
    LocationService-->>FamilyWeb: Latest location + ETA + ride status
```

- Driver app sends location every 5-10 seconds only after ride starts.
- Family tracking link must use a secure random token, not raw ride ID.
- Family viewer should see limited data: ride status, approximate live location, ETA, last updated time, and emergency/report option.
- Other riders personal details must not be exposed.
- If network is weak, show last known location and timestamp. Do not fake live movement.

9. Offline and Server Down Handling

Mobile networks can be weak during intercity rides. Driver actions must not be lost. The mobile app should use an offline action queue and the backend must process actions idempotently.

```
sequenceDiagram
    actor Driver
    participant DriverApp
    participant LocalQueue
    participant Backend
    participant BookingService

    Driver->>DriverApp: Tap critical action
    DriverApp->>LocalQueue: Save action with UUID/actionId
```

```

DriverApp-->>Backend: Try sync
alt Online
  Backend-->>BookingService: Process idempotently
  Backend-->>DriverApp: Synced
  DriverApp-->>LocalQueue: Mark synced
else Offline/server down
  DriverApp-->>Driver: Saved offline, will sync later
end
DriverApp-->>Backend: Retry pending actions later
Backend-->>BookingService: Ignore duplicate actionId if already processed

```

Critical Offline Action	Required Data
Start ride	actionId, rideId, driverId, GPS, clientTimestamp
Arrived at pickup	actionId, rideId, bookingId, stopId, GPS, clientTimestamp
Verify OTP	actionId, bookingId, OTP, GPS, clientTimestamp
Mark no-show	actionId, bookingId, waitTime, GPS, clientTimestamp
Confirm drop-off	actionId, bookingId, stopId, GPS, clientTimestamp
Finish ride	actionId, rideId, GPS, clientTimestamp

10. Payments and Payout Eligibility

The rider has already paid or authorized payment during booking. The operational flow only decides whether the booking amount becomes payout eligible, refunded, or held for dispute.

```

Rider pays during booking
-> funds held in escrow
-> pickup OTP verified
-> drop-off confirmed
-> dispute window
-> payout eligible
-> payout released to driver

```

Situation	Rider Outcome	Driver Outcome
Successful pickup and drop-off	No refund	Payout eligible after dispute window
Rider no-show	No refund or partial refund based on policy	Payout eligible after dispute window
Driver missed pickup	Full refund	No payout for that booking
Drop-off disputed	Payment held	Payout blocked until decision
One rider disputes but others are fine	Only disputed booking held	Other completed bookings can become payout eligible

11. Dispute Review System

Disputes can grow quickly, so the system should not rely on manual review for every case. Use automatic evidence collection, rule-based decisioning, risk scoring, and manual review only for unclear or high-risk cases.

```

flowchart TD
  A[Dispute raised] --> B[Freeze booking payment]
  B --> C[Evidence Collector]
  C --> D[Rule Engine]
  D --> E[Risk Score]
  E --> F{Clear case?}
  F -- Rider fault --> G[Auto release driver payout]
  F -- Driver fault --> H[Auto refund rider]
  F -- Unclear/high risk --> I[Manual admin queue]
  I --> J[Admin decision]
  J --> K[Refund / payout / split / escalate]

```

Evidence	Purpose
Driver GPS history	Was the driver near pickup/drop-off?
Rider GPS at report time	Was the rider at pickup/drop-off?
OTP status	Was onboarding actually verified?
Geofence result	Was the action near the expected location?
Timestamps	Was driver/rider on time? Did driver wait enough?
Chat/call attempts	Did parties try to communicate?

Other rider confirmations	Can other bookings support the event history?
User history	Repeated no-shows, missed pickups, false disputes, safety reports.

Dispute State	Meaning
OPEN	User raised dispute.
EVIDENCE_COLLECTED	System gathered GPS, OTP, timestamps, chat, and payment data.
AUTO_RESOLVED_RIDER_REFUND	System confidently decided driver fault.
AUTO_RESOLVED_DRIVER_PAYOUT	System confidently decided rider fault.
NEEDS_MANUAL_REVIEW	System cannot decide safely.
WAITING_FOR_USER_RESPONSE	Admin/support requested additional input.
RESOLVED_REFUND	Rider refunded.
RESOLVED_PAYOUT	Driver payout approved.
RESOLVED_SPLIT	Amount split between parties.
ESCALATED	Safety/fraud/legal/support escalation.

11.1 Auto-Resolution Examples

Case	Auto Decision Logic	Result
Rider no-show	Driver arrived within 200m, waited 10 minutes, OTP not verified, rider not near pickup, no timely dispute.	Driver payout eligible.
Driver missed pickup	Rider was within 200m and tapped I am here, driver never reached pickup, OTP not verified, rider reported within 30 minutes.	Rider refunded.
OTP verified but rider disputes pickup	OTP verified and driver GPS valid.	Likely driver-favorable unless safety issue.
Wrong drop-off	Drop-off geofence failed and rider disputes.	Manual review in MVP.

12. Edge Cases

Edge Case	System Handling
Rider does not show	Driver taps Arrived, waits required time, marks no-show; payment held during dispute window.
Driver skips rider	Rider reports missed pickup; check rider GPS, driver GPS, OTP absence; refund if driver fault.
Driver GPS unavailable	Allow action with warning, require rider confirmation, mark event suspicious.
OTP wrong	Limited retries; lock and require support/manual verification after too many failures.
Rider phone dead	Manual pickup verification; payment held until rider/support confirms.
Driver confirms wrong rider	OTP per booking prevents most cases; unresolved conflicts go to dispute.
Wrong drop-off location	Geofence fails; require rider confirmation or dispute review.
Server down	Save action locally; sync later with idempotency key.
App killed during ride	On reopen, fetch ride state, resume tracking, sync pending actions.
Driver tries to finish early	Backend rejects if any booking is still ONBOARD or pending.
Multiple riders at same pickup	Verify OTP separately for each rider.
Multiple riders at same drop-off	Drop and complete each booking separately.
Weak live tracking network	Family screen shows last known location and timestamp.
Location spoofing	Detect impossible speed/jumps, poor GPS accuracy, repeated suspicious patterns; hold payout if needed.

13. API Design

API	Purpose
POST /rides/{rideId}/start	Start the ride and begin live tracking.
POST /rides/{rideId}/finish	Finish ride if all bookings are terminal.
POST /bookings/{bookingId}/driver-arrived	Mark driver arrived at pickup and start wait timer.
POST /bookings/{bookingId}/verify-pickup-otp	Verify OTP and mark rider onboard.
POST /bookings/{bookingId}/mark-no-show	Mark rider as no-show after wait time.
POST /bookings/{bookingId}/confirm-dropoff	Driver confirms rider drop-off.
POST /bookings/{bookingId}/rider-confirm-dropoff	Rider confirms successful drop-off.
POST /bookings/{bookingId}/report-missed-pickup	Rider reports driver did not pick them up.
POST /rides/{rideId}/locations	Driver app sends GPS update.
GET /rides/{rideId}/latest-location	Rider app retrieves latest driver location.
POST /bookings/{bookingId}/tracking-link	Generate secure family tracking link.
GET /tracking/{token}	Family live tracking endpoint.
POST /offline-actions/sync	Sync locally queued actions.
POST /disputes	Create a dispute.
GET /admin/disputes	Admin dispute queue.
POST /admin/disputes/{id}/decision	Admin resolves dispute.

14. Database Model

Ride(id, driverId, routeId, status, plannedStartTime, actualStartTime, actualEndTime, currentStopSequence, createdAt, updatedAt)
Stopover(id, rideId, sequence, name, lat, lng, plannedArrivalTime, plannedDepartureTime, geofenceRadiusMeters)
Booking(id, rideId, riderId, pickupStopId, dropoffStopId, status, paymentStatus, pickupVerifiedAt, dropoffVerifiedAt, noShowMarkedAt, disputeId)
RideEvent(id, rideId, bookingId, actionId, eventType, actorType, actorId, lat, lng, clientTimestamp, serverTimestamp, validationStatus, metadataJson)
LocationUpdate(rideId, driverId, lat, lng, speed, heading, accuracy, timestamp)
OTP(id, bookingId, otpHash, expiresAt, usedAt, attemptCount, status)
TrackingLink(id, bookingId, tokenHash, expiresAt, accessScope, createdBy)
Dispute(id, rideId, bookingId, raisedBy, reason, status, evidenceJson, recommendation, riskScore, resolution, createdAt, resolvedAt)
PaymentLedger(id, bookingId, amount, currency, status, payoutEligibleAt, payoutReleasedAt, refundAt, providerReference)

15. Frontend Screens and Wireframes

15.1 Driver Screens

+-----+ TODAY'S RIDE +-----+	
Hamburg -> Tallinn Departure: 08:00 Passengers: 3 booked Status: READY TO START Stop 1 - Pickup Rider 1 Stop 2 - Pickup Rider 2, Rider 3 Stop 3 - Drop Rider 1 Stop 4 - Drop Rider 2, Rider 3 [START RIDE] +-----+	
+-----+ RIDE IN PROGRESS +-----+	
Current stop: Stop 1 Passengers onboard: 0 GPS: Active Network: Online Last synced: 09:00 	

```

| Next action: Pickup Rider 1 |
| [ ARRIVED AT PICKUP ] |
+-----+
| PICKUP RIDER |
+-----+
| Rider: Rider 1 |
| Segment: Stop 1 -> Stop 3 |
| Distance to pickup: 50m |
| Geofence: Valid |
| Enter OTP: [ _ _ _ _ ] |
| [ VERIFY OTP ] [ MARK NO-SHOW ] |
+-----+
| DROP-OFF |
+-----+
| Rider: Rider 1 |
| Destination: Stop 3 |
| Geofence: Valid |
| [ CONFIRM DROP-OFF ] |
+-----+
| RIDE SUMMARY |
+-----+
| Completed: 3 riders |
| No-show: 0 |
| Disputes: 0 |
| Expected earnings: EUR 42 |
| [ FINISH RIDE ] |
+-----+

```

15.2 Rider Screens

```

+-----+
| UPCOMING RIDE |
+-----+
| Hamburg -> Tallinn |
| Pickup: Stop 2 at 09:00 |
| Driver: Michael |
| Vehicle: BMW 320 |
| [ LIVE TRACKING ] |
| [ SHARE TRACKING ] |
+-----+
| DRIVER APPROACHING |
+-----+
| ETA: 5 minutes |
| Driver: Michael |
| Vehicle: BMW 320 |
| Your OTP: 8437 |
| [ I'M AT PICKUP ] |
| [ DRIVER DID NOT PICK ME UP ] |
+-----+
| ONBOARD |
+-----+
| Status: Ride in progress |
| ETA to destination: 2h 15m |
| [ MAP / LIVE LOCATION ] |
| [ REPORT ISSUE ] |
+-----+
| ARRIVED |
+-----+
| Driver marked you as dropped off. |
| Did you arrive safely? |

```

```

| [ YES, CONFIRM ]
| [ REPORT ISSUE ]
+-----+

```

15.3 Family Live Tracking Screen

```

+-----+
| LIVE TRIP
+-----+
| Passenger: Rider 1
| Status: ONBOARD
| Current location: Map
| ETA: 1h 45m
| Last updated: 10 seconds ago
|
| [ EMERGENCY / REPORT ]
+-----+

```

16. Admin Dashboard

```

+-----+
| DISPUTE REVIEW
+-----+
| Ride: R123           Booking: B456
| Amount: EUR 18
| Reason: Driver did not pick me up
|
| Recommendation: Refund rider
| Confidence: 87%
|
| Evidence:
| - Rider GPS valid at pickup
| - Driver GPS never reached pickup
| - OTP not verified
| - Rider reported within 30 minutes
|
| [ REFUND RIDER ] [ PAY DRIVER ] [ SPLIT ]
| [ ASK DRIVER ] [ ASK RIDER ] [ ESCALATE ]
+-----+

```

- Admin list should be priority based, not a flat list.
- High priority: safety issues, high-value rides, repeated offenders, payment blocked for more than SLA.
- Each dispute should show system recommendation, confidence score, evidence timeline, and one-click resolution buttons.

17. MVP Scope and Future Enhancements

MVP	Future
OTP pickup verification	Automatic pickup/drop-off detection
Driver GPS every 10 seconds	Advanced route deviation alerts
Secure family tracking link	Real-time fraud/ML risk scoring
Geofence pickup/drop-off validation	Dynamic rerouting and stop optimization
Offline action queue	Predictive no-show scoring
Basic dispute rule engine	Advanced trust score model
Manual admin review queue	Automated support chat workflows

18. Development Checklist

- Implement ride state machine and strict transition validation.
- Implement booking state machine per rider booking.
- Implement OTP generation, display, verification, expiry, and retry limit.
- Implement driver stop timeline UI.

- Implement live location upload and latest location retrieval.
- Implement secure family tracking link with expiry.
- Implement geofence validation for pickup and drop-off.
- Implement offline action queue in driver app.
- Implement idempotent backend action processing using actionId.
- Implement payment ledger per booking.
- Implement dispute creation and payment hold.
- Implement evidence collector and rule-based auto-resolution.
- Implement admin dispute review dashboard.
- Implement notifications for all important lifecycle events.
- Add audit logs for all state transitions and payment decisions.
- Add monitoring for failed syncs, suspicious GPS, stuck bookings, and pending payouts.